

Learning Neural Networks in TensorFlow Playground

Applied Deep Learning — Chapter 2

Yin Yang

Foundation Books Project

Roadmap

The Spiral Laboratory

One Neuron, Many Layers

Training Loop and Loss

Learning Rate and Activations

Regularization

Guided Session

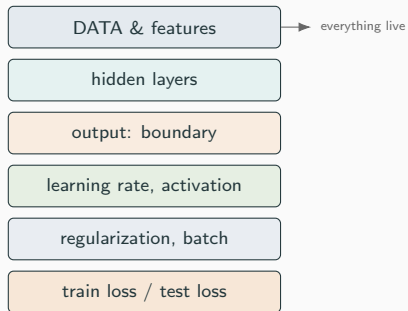
Practical Checklist

Summary

The Spiral Laboratory

Why a Visible Laboratory?

- TensorFlow Playground at `playground.tensorflow.org` renders an entire neural network on one screen.
- Goal: **see, name, and reason about** what a network is doing while it learns — before any code.
- Spiral classification: two arms wind around the origin, so no straight boundary can work.
- Watch three things together: data dots, colored background (classifier), loss numbers.



Anatomy of a Classification Problem

Definitions

A *labeled dataset* is a collection of examples (x_i, y_i) . *Binary classification* assigns each input to one of two classes.

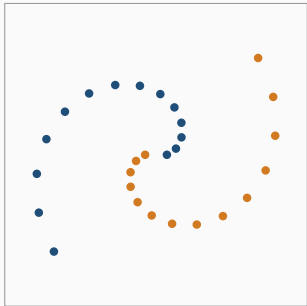
In the spiral activity:

- Input (u, v) : a point in $\mathbb{R}^2$.
- Label: blue or orange.
- Model: $f : \mathbb{R}^2 \rightarrow \{\text{blue}, \text{orange}\}$.

The *decision boundary* is where the predicted class changes — the place the colored background flips.

Spiral Geometry

two classes wind around the origin



goal: a
boundary that
bends with
the arms

A single straight split cuts across both arms. The need for hidden layers and nonlinearity is visible before any equation.

Train vs. Test, Loss vs. Accuracy

Training data examples used to adjust the network's parameters.

Test data held-out examples used to estimate generalization.

Accuracy fraction of examples classified correctly.

Loss numerical penalty for wrong or uncertain predictions — what training actually watches.

Playground shows both training loss and test loss because coloring the training points right is only part of the job.

One Neuron, Many Layers

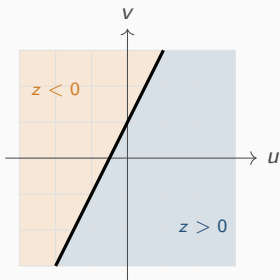
A Neuron is a Small Trainable Formula

Inputs $x_1, \dots, x_d$, weights $w_1, \dots, w_d$, bias b :

$$z = w_1x_1 + \dots + w_dx_d + b, \quad a = \phi(z).$$

- Training changes **weights and biases**.
- Training does **not** change inputs or labels.
- Each circle in a Playground hidden layer is one such neuron, with a small map of where in the input plane it responds.

One Linear Neuron Draws One Line



With

$$w_1 = 2, w_2 = -1, b = 0.5, \phi(z) = z:$$

$$z = 2u - v + 0.5.$$

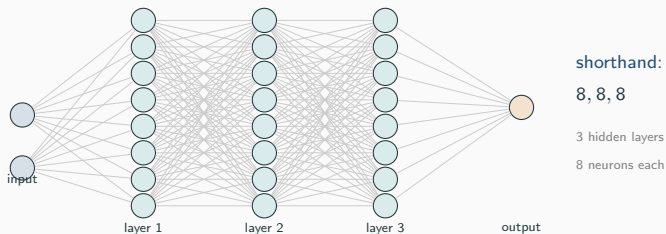
The line $v = 2u + 0.5$ is the boundary.

A linear neuron $\Rightarrow$ one straight split.
The spiral needs a boundary that *bends*.

Layers and Architecture

Definition

A *layer* is a group of neurons that share an input stage. The *architecture* is the count and width of layers.



Representation learning: raw coordinates → broad regions → recombined parts → output decision.

Connections, Weights, Learning

In Playground:

- Connection *color* = sign of weight.
- Connection *thickness* = magnitude of weight.

Concrete arithmetic: source neuron output $a = 0.7$, weight $w = 2.0$, contribution to next sum is $wa = 1.4$. Train it down to $w = -1.0$ and the same a now contributes -0.7 .

Learning = many small numbers in connections and biases moving so that predictions agree better with labels.

Training Loop and Loss

Epochs, Batches, and the Loop

Definitions

An *epoch* is one pass through the training data. The *batch size* is how many examples contribute to one parameter update.

- Small batch → many noisy updates per epoch.
- Large batch → smoother but possibly slower visible change.
- Epoch counts repeated exposure to the data.

Binary Cross-Entropy

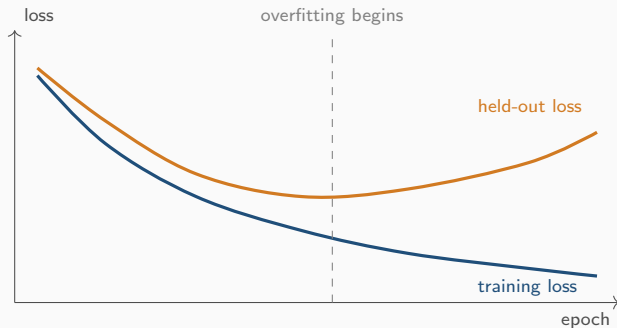
For label $y_i \in \{0, 1\}$ and predicted probability $\hat{p}_i$:

$$\ell_i = -y_i \log(\hat{p}_i) - (1 - y_i) \log(1 - \hat{p}_i),$$

$$\text{training loss} = \frac{1}{n} \sum_{i=1}^n \ell_i.$$

Playground hides the formula behind a live number, but the number still means **average prediction penalty per example**.

Underfitting and Overfitting



- **Underfitting:** broad boundary cuts across the spiral arms.
- **Overfitting:** jagged islands around noisy points; held-out loss rises while training loss keeps falling.
- Watching training loss alone is not enough.

Learning Rate and Activations

Learning Rate as Step Size

Update rule (preview):

$$w_{\text{new}} = w - \eta \cdot (\text{update signal}).$$

Worked example: $w = 0.80$, signal 0.30.

- $\eta = 0.10$ gives $w_{\text{new}} = 0.77$.
- $\eta = 0.01$ gives $w_{\text{new}} = 0.797$.

Diagnosis:

Loss barely moves step too small.

Loss bounces / boundary thrashes step too large.

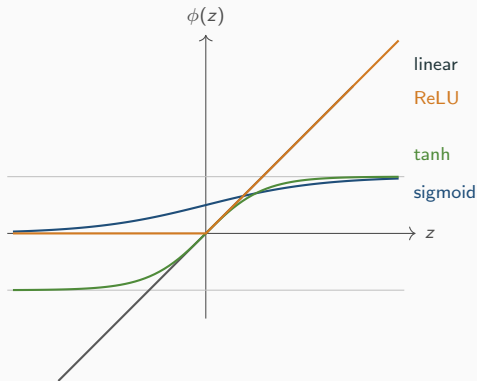
Steady descent then plateau try a manual schedule.

Manual Learning-Rate Schedule (E2 example)

Stage	Action	Rate	Epoch	Held-out loss
Warm start	reset, press play	0.10	308	0.199
Fine adjustment	pause, lower rate	0.03	909	0.087

Pattern: **warm start**, **stabilize**, **fine tune**. A small rate later helps the boundary settle without overshooting.

Four Activations on One Axis



- **Linear**: stack of linear layers $\equiv$ one linear rule. Cannot bend.
- **Sigmoid, tanh**: bounded, smooth, can saturate.
- **ReLU**: $\max(0, z)$; sharp pieces, may leave units inactive.

Comparing Activations Fairly

Fix dataset, split, seed, architecture, batch size, regularization, schedule. Change *only* the activation.

Activation	Spiral boundary	Note
Linear	broad transition	cannot bend
Sigmoid	smooth, slow to learn	saturation slows updates
Tanh	smooth spiral forms	solid baseline for 2D toy
ReLU	sharp piecewise spiral	fastest in the recorded run

No universal winner. Tie the claim to the run on the screen, not to all neural networks.

Regularization

Regularization as a Weighted Score

regularized score = prediction loss + λ · complexity penalty.

- L2 prefers smaller weights overall.
- L1 also penalizes weight size; can drive some connections to effectively zero.
- The slider in Playground sets λ .

Diagnose first. Add regularization after observing overfitting or noisy-data jaggedness, not because the menu offers it.

Noise + L2: A Worked Comparison

Same architecture (tanh, 8, 8, 8), same epoch budget, noise 20.

Run	Train loss	Held-out loss	Boundary
E5: no regularization	0.014	0.102	jagged near noisy points
E6: L2, rate 0.001	0.029	0.077	smoother, spiral preserved

L2 **raised** training loss but **lowered** held-out loss — the useful tradeoff regularization offers when correctly diagnosed.

Guided Session

Six Configurations, One Protocol

ID	Purpose	Configuration
E1	weak baseline	linear, 2 hidden units, no reg, noise 0
E2	viable model + schedule	tanh, 8, 8, 8, schedule 0.1 $\rightarrow$ 0.03
E3	sigmoid comparison	sigmoid, 8, 8, 8, lr 0.1
E4	ReLU comparison	ReLU, 8, 8, 8, lr 0.1
E5	noisy, no regularization	tanh, 8, 8, 8, noise 20
E6	noisy, mild L2	tanh, 8, 8, 8, noise 20, L2 0.001

E1 $\rightarrow$ E2 is a *diagnostic rebuild* (many controls change at once); E3, E4, E5, E6 are *controlled comparisons*.

Recorded Results

ID	Train	Held-out	Boundary observation
E1	0.452	0.498	broad transition; underfit
E2	0.002	0.087	spiral-like with some center confusion
E3	0.448	0.493	broad smooth regions; sigmoid underfits
E4	0.000	0.011	sharp, follows spiral closely
E5	0.014	0.102	follows noisy spiral but jagged
E6	0.029	0.077	smoother, spiral preserved

“It got better” is not a conclusion. What the boundary shows under a recorded setup is the conclusion.

Practical Checklist

1. Start from problem geometry — spiral needs a curved boundary.
2. Use only raw coordinates first; force the hidden layers to learn.
3. Begin simple; add width or depth only when underfit is visible.
4. Compare activations under controlled settings.
5. Read training loss, held-out loss, and boundary shape *together*.
6. Treat learning rate as step size; warm start then fine tune.
7. Add regularization only after diagnosis.
8. Keep a run log: configuration, epochs, losses, one sentence on boundary shape.

What Belongs in a Run Log

- dataset and features (so the boundary solves the same problem)
- hidden layers and activation (how the boundary is allowed to bend)
- learning rate and regularization (possible cause of instability or smoothing)
- epoch count, training loss, held-out loss
- one sentence about boundary shape

Useful: “tanh, 8, 8, lr 0.03, no reg; held-out loss stopped improving once boundary became wavy.”

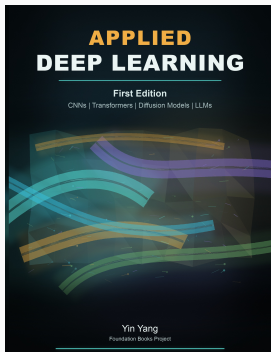
Not useful: “it got better.”

Summary

Summary

- Spiral classification makes the **need** for nonlinear boundaries visible before any code.
- A neuron computes $a = \phi(w^\top x + b)$. Training adjusts weights and biases, not data or labels.
- Layers are stages of representation learning; depth without nonlinearity collapses to one linear rule.
- Loss is the live training signal; held-out loss judges generalization. Read both together with the boundary.
- Learning rate is step size; activation choice decides what shapes the boundary can take; regularization trades fit for smoothness.
- Discipline: change one thing at a time, record links and losses, let the boundary tell you what to change next.

Next: write the same habits in Python — MNIST with Keras and PyTorch.



Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/applied-deep-learning
Code	github.com/foundation-books/applied-deep-learning-code
Print	amazon.com/dp/B0GZF7QRSH
Kindle	amazon.com/dp/B0GZF9221X
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.